

Market Risk VaR (Value at Risk) Estimation

0% Paid API — 100% Free & Local Tools Only

Domain	Market Risk Management
Dataset	Free daily price history for any portfolio of tickers via yfinance
Source Link	https://finance.yahoo.com
Skills Used	Python, NumPy, Monte Carlo simulation, yfinance, pandas

Problem Statement

Risk desks need a daily estimate of maximum expected portfolio loss at a given confidence level. Build a VaR engine that computes Historical, Parametric, and Monte Carlo VaR for a multi-asset portfolio -- using only free market data and open-source numerical libraries.

What You Will Build

- A free price history loader for a multi-ticker portfolio via yfinance
- Daily return calculation and a portfolio-weighted return series
- Historical VaR (empirical percentile method)
- Parametric VaR (variance-covariance / normal distribution method)
- Monte Carlo VaR (simulated return paths using the portfolio's covariance matrix)
- A comparison chart of all three VaR estimates at 95% and 99% confidence

Step-by-Step Guide

Step 1: Load Free Price Data

WHAT: Pull daily adjusted close prices for the portfolio tickers via yfinance

WHY: yfinance gives free daily data adequate for VaR estimation without a paid market data feed

HOW: `yfinance.download(tickers, period='2y')['Close']`

Step 2: Compute Portfolio Returns

WHAT: Calculate daily log returns and weight them by portfolio allocation

WHY: VaR is calculated on the portfolio's combined return series, not each asset separately

HOW: `returns = np.log(prices/prices.shift(1)); portfolio_returns = (returns * weights).sum(axis=1)`

Step 3: Historical & Parametric VaR

WHAT: Compute the empirical percentile VaR and the normal-distribution VaR

WHY: Comparing both methods highlights how much fat-tail risk the normal assumption misses

HOW: `np.percentile(returns, 5)` for historical; `mean - z*std` for parametric

Step 4: Monte Carlo VaR

WHAT: Simulate thousands of correlated return paths using the covariance matrix

WHY: Monte Carlo captures correlation structure and can model more complex distributions than parametric VaR

HOW: `np.random.multivariate_normal(mean_returns, cov_matrix, n_simulations)`

Complete Solution Code (100% Free — No Paid API Keys)

```
# pip install yfinance numpy pandas matplotlib --break-system-packages

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf

# 1. LOAD FREE PRICE DATA
tickers = ["AAPL", "MSFT", "JPM", "XOM"]
weights = np.array([0.3, 0.3, 0.2, 0.2])
portfolio_value = 1_000_000

prices = yf.download(tickers, period="2y")["Close"]

# 2. COMPUTE PORTFOLIO RETURNS
log_returns = np.log(prices / prices.shift(1)).dropna()
portfolio_returns = log_returns.dot(weights)

mean_return = portfolio_returns.mean()
std_return = portfolio_returns.std()

# 3. HISTORICAL VaR (empirical percentile method)
def historical_var(returns, confidence=0.95):
    return -np.percentile(returns, (1 - confidence) * 100)

hist_var_95 = historical_var(portfolio_returns, 0.95) * portfolio_value
hist_var_99 = historical_var(portfolio_returns, 0.99) * portfolio_value

# 4. PARAMETRIC VaR (variance-covariance, normal distribution)
from scipy.stats import norm
def parametric_var(mean, std, confidence=0.95):
    z = norm.ppf(1 - confidence)
    return -(mean + z * std)

param_var_95 = parametric_var(mean_return, std_return, 0.95) * portfolio_value
param_var_99 = parametric_var(mean_return, std_return, 0.99) * portfolio_value

# 5. MONTE CARLO VaR
cov_matrix = log_returns.cov()
mean_vec = log_returns.mean().values
n_sims = 20000

simulated_returns = np.random.multivariate_normal(mean_vec, cov_matrix, n_sims)
simulated_portfolio_returns = simulated_returns.dot(weights)

mc_var_95 = -np.percentile(simulated_portfolio_returns, 5) * portfolio_value
mc_var_99 = -np.percentile(simulated_portfolio_returns, 1) * portfolio_value

# 6. COMPARE ALL THREE METHODS
results = pd.DataFrame({
    "Method": ["Historical", "Parametric", "Monte Carlo"],
    "VaR 95% ($)": [hist_var_95, param_var_95, mc_var_95],
    "VaR 99% ($)": [hist_var_99, param_var_99, mc_var_99],
})
print(results)

plt.figure(figsize=(9, 5))
plt.hist(simulated_portfolio_returns * portfolio_value, bins=80, alpha=0.6, label="Simulated P&L")
plt.axvline(-mc_var_95, color="orange", linestyle="--", label="95% VaR")
plt.axvline(-mc_var_99, color="red", linestyle="--", label="99% VaR")
plt.legend(); plt.title("Monte Carlo Simulated Portfolio P&L Distribution")
plt.tight_layout(); plt.savefig("var_distribution.png", dpi=150); plt.show()

results.to_csv("var_comparison.csv", index=False)
```

Free Dashboard / Reporting Ideas

- KPI cards: 95% and 99% VaR side by side for each method (Historical/Parametric/Monte Carlo)

- Histogram: Monte Carlo simulated P&L; distribution with VaR lines marked
- Line chart: rolling 30-day VaR over the trailing year to track risk trend
- Table: per-asset contribution to total portfolio variance
- Alert card: flag if today's actual loss exceeds the 95% VaR estimate (backtesting exception)

Key Insights

- Parametric VaR consistently understates tail risk versus Historical/Monte Carlo when returns aren't normally distributed
- Monte Carlo VaR properly captures cross-asset correlation, which simple historical VaR on a blended series can miss
- VaR should always be backtested: count how often actual losses exceed the VaR threshold and compare to the confidence level
- yfinance is sufficient for portfolio-level VaR estimation at the retail/educational scale without any paid data feed
- This is the same three-method VaR comparison risk desks use to sanity-check any single model's assumptions

Tools (all free): Python 3 | yfinance | NumPy | SciPy | pandas